

Dokumentation DistoFileErzeugen.xlsm

1 INHALTSVERZEICHNIS

2	Gesamtüberblick.....	2
3	Architektur	2
3.1	Aufbau der Anwendung	2
3.2	Datenfluss	2
4	Einstiegspunkt.....	2
4.1	Main-Modul	2
5	Benutzeroberflächen	2
5.1	PowerShell-Form (PWAI1F)	3
5.2	VBA-Form (VBAAI1F)	3
6	Datenverarbeitung.....	4
6.1	Einlesen der Daten.....	4
6.2	Trennung in Ana und Digi	4
6.3	Zusammenführung der Daten.....	4
6.4	CSV-Erstellung	4
7	PowerShell-Integration	5
7.1	Ablauf.....	5
7.2	Parallelisierung.....	5
8	Dateiverwaltung	5
8.1	Auswahlmechanismen.....	5
8.2	Ordnerstruktur	5
8.3	Rekursive Verarbeitung	5
9	Hilfsfunktionen.....	6
9.1	Pfadverarbeitung	6
9.2	Dateivergleich.....	6
9.3	Ordnererstellung.....	6
9.4	Debugging	6
10	Designentscheidungen	6
10.1	Dualer Verarbeitungsansatz	6
10.2	Einsatz von Dictionaries	6
10.3	Arraybasierte Verarbeitung.....	6
10.4	Rekursive Verarbeitung	6
11	Zusammenfassung.....	7

2 GESAMTÜBERBLICK

Die vorliegende Anwendung ist eine Excel-basierte Lösung zur strukturierten Verarbeitung von Datendateien. Sie bietet zwei alternative Verarbeitungsansätze: eine vollständig in VBA implementierte Variante sowie eine Performanz optimierte Variante, die externe PowerShell-Prozesse nutzt. Beide Wege verfolgen denselben fachlichen Ablauf, unterscheiden sich jedoch in der technischen Umsetzung und Ausführungsgeschwindigkeit.

Im Kern dient die Anwendung dazu, zwei zusammengehörige Dateien – eine sogenannte Dummy-Datei und eine Originaldatei – einzulesen, deren Inhalte zu analysieren, logisch zu trennen und anschließend in definierter Form als CSV-Dateien auszugeben. Neben der Einzelverarbeitung unterstützt das System auch die automatisierte Verarbeitung ganzer Ordnerstrukturen.

3 ARCHITEKTUR

3.1 AUFBAU DER ANWENDUNG

Die Architektur ist modular aufgebaut und trennt klar zwischen Benutzerinteraktion, Datenverarbeitung und technischen Hilfsfunktionen. Die Benutzerinteraktion erfolgt über zwei UserForms, die jeweils eine der beiden Verarbeitungsstrategien repräsentieren. Die eigentliche Logik ist in mehrere spezialisierten Module ausgelagert, wodurch eine Wiederverwendung und bessere Wartbarkeit ermöglicht wird.

Die Verarbeitung folgt dabei einem konsistenten Schema: Eingabedaten werden eingelesen, in interne Datenstrukturen überführt, anschließend analysiert und transformiert und schließlich in ein Ausgabeformat geschrieben.

3.2 DATENFLUSS

Der Ablauf beginnt mit der Auswahl von Eingabedateien oder -ordnern. Die Daten werden anschließend in Arrays eingelesen und in Dictionaries organisiert. Darauf folgt eine Trennung in zwei logische Bereiche (Ana und Digi), die jeweils separat weiterverarbeitet werden. Nach der Zusammenführung relevanter Inhalte entstehen zwei Zielstrukturen, die schließlich in CSV-Dateien geschrieben werden.

4 EINSTIEGSPUNKT

4.1 MAIN-MODUL

Das Main-Modul stellt den zentralen Einstiegspunkt der Anwendung dar. Die darin enthaltene Prozedur Main übernimmt die Initialisierung, indem sie Standardpfade relativ zum Speicherort der Excel-Datei festlegt. Diese Pfade werden direkt in den Benutzeroberflächen angezeigt, sodass der Benutzer unmittelbar mit einer konsistenten und vorgegebenen Ordnerstruktur arbeiten kann.

Anschließend wird die VBA-basierte Benutzeroberfläche geladen und angezeigt. Die PowerShell-Variante ist ebenfalls bereits integriert und kann über die Benutzeroberfläche ausgewählt werden. Diese initiale Konfiguration stellt sicher, dass alle weiteren Verarbeitungsschritte auf einer klar definierten Verzeichnisstruktur basieren.

Darüber hinaus wird im Main-Modul gesteuert, welche Ansicht beim Programmstart sowie beim Wechsel zwischen VBA- und PowerShell-Bearbeitung standardmäßig aktiv ist. Dabei kann zwischen der Einzeldatei- und der Mehrfachdatei-Bearbeitung unterschieden werden. Die Auswahl erfolgt über zwei String-Variablen (StartPunktVBA und StartPunktPW), die global definiert sind und somit das Verhalten der gesamten Anwendung beeinflussen.

Zusätzlich können in diesem Modul zentrale Einstellungen angepasst werden, die im weiteren Programmverlauf verwendet werden. Dazu zählen unter anderem:

- die maximale Anzahl parallel laufender PowerShell-Prozesse,
- die Startzeile, ab der Daten eingelesen werden,
- die maximale Anzahl an zu verarbeitenden Spalten,
- sowie die Sichtbarkeit der PowerShell-Instanzen auf der Benutzeroberfläche (z. B. ausgeblendet, minimiert oder maximiert).

Diese zentral gebündelten Parameter ermöglichen eine einfache Anpassung des Programmverhaltens und dienen als grundlegende Konfigurationsbasis der Anwendung.

5 BENUTZEROBERFLÄCHEN

5.1 POWERSHELL-FORM (PWA11F)

Diese Benutzeroberfläche dient der Steuerung der externen Verarbeitung über PowerShell. Sie übernimmt die Auswahl von Dateien und Ordnern, verwaltet Statusanzeigen und startet sowohl Einzel- als auch Mehrfachverarbeitungen.

Bei der Einzelverarbeitung wird zunächst geprüft, ob beide erforderlichen Dateien vorhanden sind. Anschließend wird ein temporäres PowerShell-Skript erzeugt, ausgeführt und nach Abschluss wieder gelöscht. Die eigentliche Datenverarbeitung findet vollständig außerhalb von VBA statt.

Die Mehrfachverarbeitung basiert auf einer rekursiven Durchsuchung von Ordnerstrukturen. Dabei werden Unterordner systematisch durchlaufen und passende Dateipaare anhand ihrer Dateinamen (ohne Erweiterung) identifiziert. Für jedes gefundene Paar wird ein separater PowerShell-Prozess gestartet.

Ein wesentliches Merkmal dieser Variante ist die Parallelisierung. Es werden mehrere Prozesse gleichzeitig gestartet, wobei eine Obergrenze definiert ist, um Systemressourcen zu kontrollieren.

5.2 VBA-FORM (VBA11F)

Diese Variante bildet den vollständigen Verarbeitungsprozess innerhalb von VBA ab. Sie bietet funktional denselben Ablauf wie die PowerShell-Version, verzichtet jedoch vollständig auf externe Abhängigkeiten.

Die Einzelverarbeitung erfolgt sequenziell. Nach dem Einlesen der Daten werden diese in zwei Bereiche getrennt, anschließend zusammengeführt und schließlich exportiert. Jeder dieser Schritte ist in ein eigenes Modul ausgelagert.

Die Mehrfachverarbeitung nutzt ebenfalls eine rekursive Ordnerdurchsuchung. Im Gegensatz zur PowerShell-Variante erfolgt die Verarbeitung jedoch direkt innerhalb von VBA, ohne parallele Prozesse.

6 DATENVERARBEITUNG

6.1 EINLESEN DER DATEN

Das Einlesen erfolgt im Modul VBAExcelStroerschInDictArray. Hier wird eine zweistufige Strategie verwendet. Zunächst wird die Anzahl der relevanten Zeilen ermittelt, um die benötigten Arrays exakt zu dimensionieren. Anschließend werden die Daten zeilenweise eingelesen und in eine strukturierte Datenform überführt.

Die interne Speicherung erfolgt zunächst in einem benutzerdefinierten Datentyp mit festen Arrays pro Spalte. Erst danach werden diese Daten in ein Dictionary überführt, um eine flexible Weiterverarbeitung zu ermöglichen. Diese Kombination erlaubt eine hohe Performance bei gleichzeitig einfacher Handhabung.

6.2 TRENNUNG IN ANA UND DIGI

Die Aufteilung der Daten erfolgt im Modul VBATrennungDictArray. Grundlage ist der Verlauf einer ID-Spalte. Sobald ein Rückgang der ID erkannt wird, wird dies als Übergangspunkt interpretiert.

Alle Daten vor diesem Punkt werden dem Ana-Bereich zugeordnet, alle folgenden dem Digi-Bereich. Diese Logik ermöglicht eine klare Trennung ohne zusätzliche Marker in den Daten.

6.3 ZUSAMMENFÜHRUNG DER DATEN

Die Zusammenführung wird im Modul VBAAnaDigiDictArray umgesetzt. Hier werden aus den getrennten Daten zwei neue Strukturen erzeugt: AnaText und DigiText.

Dabei werden nur relevante Spalten übernommen und gegebenenfalls ergänzt. Zusätzlich werden Standardwerte gesetzt, wenn bestimmte Felder nicht vorhanden sind. Die Länge der resultierenden Arrays wird dynamisch bestimmt, um nur tatsächlich vorhandene Daten zu berücksichtigen.

6.4 CSV-ERSTELLUNG

Die Ausgabe erfolgt im Modul VBACSV. Die Daten werden zeilenweise in CSV-Dateien geschrieben, wobei die Spaltenüberschriften automatisch aus den Dictionary-Schlüsseln generiert werden.

Vor der Erstellung wird sichergestellt, dass der Zielordner existiert. Zudem werden ungültige Zeichen im Dateinamen entfernt, um Fehler beim Schreiben zu vermeiden.

7 POWERSHELL-INTEGRATION

7.1 ABLAUF

Das Modul PWPowerShellRunner übernimmt die vollständige Steuerung der externen Verarbeitung. Es erzeugt ein temporäres Skript, das alle notwendigen Verarbeitungsschritte enthält, und startet dieses anschließend als separaten Prozess.

Das Skript selbst implementiert eine vergleichbare Logik wie die VBA-Variante: Einlesen, Trennung, Transformation und CSV-Erstellung. Die Verarbeitung erfolgt dabei vollständig unabhängig von Excel.

7.2 PARALLELISIERUNG

Ein zentraler Aspekt ist die parallele Ausführung mehrerer Prozesse. Vor dem Start eines neuen Prozesses wird geprüft, wie viele PowerShell-Prozesse aktuell aktiv sind. Erst wenn die definierte Obergrenze unterschritten ist, wird ein weiterer Prozess gestartet.

Nach dem Start aller Prozesse wartet das System, bis alle abgeschlossen sind, bevor das Skript gelöscht wird. Dieses Vorgehen ermöglicht eine deutliche Beschleunigung bei großen Datenmengen.

8 DATEIVERWALTUNG

8.1 AUSWAHLMECHANISMEN

Die Module PWDateiAuswahl und PWOrdnerAuswahl kapseln die Auswahl von Dateien und Ordnern. Sie bieten die Möglichkeit, mit einem vordefinierten Startpfad zu arbeiten, der relativ zur Excel-Datei gesetzt wird.

8.2 ORDNERSTRUKTUR

Die Anwendung basiert auf einer festen Grundstruktur mit den Ordnern „Input“ und „Output“. Innerhalb des Input-Ordners wird zwischen „Dummy“ und „Org“ unterschieden. Diese Struktur wird sowohl bei der Einzel- als auch bei der Mehrfachverarbeitung verwendet.

8.3 REKURSIVE VERARBEITUNG

Bei der Mehrfachverarbeitung werden alle Unterordner eines Startverzeichnisses durchlaufen. Für jeden Ordner wird geprüft, ob entsprechende Gegenstücke im anderen Eingabepfad existieren. Die Verarbeitung erfolgt nur für passende Kombinationen, wodurch fehlerhafte Zuordnungen vermieden werden.

9 HILFSFUNKTIONEN

9.1 PFADVERARBEITUNG

Funktionen wie BuildPath und TrimLeadingSlash sorgen für konsistente und fehlerfreie Pfadoperationen. Sie stellen sicher, dass Verzeichnisse korrekt zusammengesetzt werden.

9.2 DATEIVERGLEICH

Die Funktion RemoveExtension ermöglicht den Vergleich von Dateien unabhängig von ihrer Erweiterung. Dies ist entscheidend für das Matching von Dummy- und Org-Dateien.

9.3 ORDNERERSTELLUNG

Die Erstellung von Ordnern erfolgt dynamisch und bei Bedarf rekursiv. Dadurch ist sichergestellt, dass alle notwendigen Zielverzeichnisse existieren, bevor Dateien geschrieben werden.

9.4 DEBUGGING

Das Modul VBAProjektDebug bietet Funktionen zur Analyse der Datenstrukturen. Inhalte von Dictionaries können entweder im Direktfenster ausgegeben oder in ein Excel-Arbeitsblatt geschrieben werden. Dies erleichtert die Fehlersuche und Validierung der Verarbeitungsschritte.

10 DESIGNENTSCHEIDUNGEN

10.1 DUALER VERARBEITUNGSANSATZ

Die Trennung in VBA- und PowerShell-Verarbeitung ermöglicht eine flexible Nutzung. Während die VBA-Variante einfacher und unabhängiger ist, bietet die PowerShell-Variante Vorteile bei großen Datenmengen durch Parallelisierung.

10.2 EINSATZ VON DICTIONARIES

Dictionaries werden verwendet, um Daten strukturiert und flexibel zu speichern. Sie ermöglichen den Zugriff über Schlüssel und erleichtern die Weitergabe zwischen Modulen.

10.3 ARRAYBASIERTE VERARBEITUNG

Die konsequente Nutzung von Arrays reduziert Speicheroperationen und vermeidet langsame Zellzugriffe. Dies ist insbesondere bei großen Datenmengen entscheidend für die Performance.

10.4 REKURSIVE VERARBEITUNG

Die rekursive Durchsuchung von Ordnern erlaubt eine skalierbare Verarbeitung beliebiger Verzeichnisstrukturen. Dadurch ist die Anwendung unabhängig von der Tiefe oder Komplexität der Eingabedaten.

11 ZUSAMMENFASSUNG

Die Anwendung stellt eine leistungsfähige und modular aufgebaute Lösung zur Datenverarbeitung dar. Sie kombiniert effiziente Datenstrukturen, klare Trennung von Verantwortlichkeiten und flexible Ausführungsoptionen.

Durch die Wahl zwischen VBA- und PowerShell-Verarbeitung kann sie sowohl für kleinere als auch für große Datenmengen eingesetzt werden. Der strukturierte Aufbau ermöglicht zudem eine einfache Erweiterung und Anpassung an neue Anforderungen.